

DWRS & HDW Load Balancer

Complete Theory + Code Guide

Dynamic Weighted Round-Robin Scheduling · Hash-IP · Dynamic Switching Path

Based on: Osei Kofi & Ahene, PLOS ONE 18(4) 2023

Contents

Section	Topic
1	What is Load Balancing? (The Basics)
2	DWRS — Dynamic Weighted Round-Robin Scheduling
3	Hash-IP — Fingerprinting Requests
4	DSP — Dynamic Switching Path (IGRP / EGRP)
5	HDW — Full Pipeline (Hash + Weighted + DSP)
6	The 4 Equations from the Paper
7	Code Walkthrough — Every File Explained
8	Algorithm Comparison Table
9	Scenario Q&A — Exam-style Questions

1. What is Load Balancing? (Starting From Zero)

Imagine you run a very popular pizza restaurant. On a normal day, one chef handles all orders fine. But on Friday night, 500 orders come in at once — that one chef gets overwhelmed, orders are delayed, and customers leave angry. The solution: hire 3 chefs and split the orders between them. **That is load balancing** — distributing incoming work across multiple servers so no single server gets overwhelmed.

Load Balancer = the smart manager who decides which chef (server) gets each new pizza order (request).

Server = one chef. Has a maximum speed (throughput) and can only hold so many orders at once (queue capacity).

Request = one incoming pizza order. Has a source (who ordered), destination (which branch), and size (how complex).

Queue = the list of pending orders a chef still needs to handle.

Without load balancing, the first server receives 100% of traffic, crashes, and all users experience downtime. With load balancing, traffic spreads intelligently — faster servers handle more, slower servers handle less.

Why not just use Round-Robin (the simplest method)?

Basic Round-Robin sends request 1 to Server A, request 2 to Server B, request 3 to Server C, then repeats. It treats all servers as equals — but they are NOT equal. Server A might be a slow 100 Mbps machine while Server C is a fast 1000 Mbps machine. Sending them the same number of requests is unfair and inefficient.

Simple Round-Robin (bad)

- Sends 1 request to each server in turn
- Ignores server speed differences
- Fast servers sit idle while slow server drowns
- No concept of "this packet is bigger"

Weighted Scheduling (better)

- Faster server = more requests assigned
- Considers queue depth (how busy a server is)
- Considers packet size (bigger = heavier)
- Result: all servers finish work at roughly the same time

2. DWRS — Dynamic Weighted Round-Robin Scheduling

DWRS stands for **D**ynamic **W**eighted **R**ound-Robin **S**cheduling. It is an algorithm that decides which server gets each incoming request, based on two key factors:

- **How fast is the server?** (its link capacity / bandwidth)
- **How large is the packet?** (its data size)

The core intuition:

Think of servers as workers with different speeds. A worker who types 100 words/min should handle fewer tasks than one who types 1000 words/min. But also — if the task is writing a short email, even the slow worker is fine. If the task is a full novel, only the fast worker should get it. DWRS assigns tasks (packets) to workers (servers) based on BOTH the worker's speed AND the task's size.

Key Terms in DWRS:

Concept	Meaning
m_i (mean packet size)	Average size of packets this server has processed. Updated live as requests arrive. Used in
l_i (link capacity share)	What fraction of total bandwidth does server i have? If server A has 100 Mbps and total is 1000 Mbps, $l_i = 0.1$.
w_i (scheduling weight)	The final scheduling priority of server i . Computed as l_i / m_i . Higher weight = more requests
bandwidth allocation	What share of bandwidth each server ACTUALLY gets based on weights and packet sizes.
deficit counter	A running credit score per server. Server with highest credit wins the next request. After win

Why "Dynamic"?

The word Dynamic is critical. Classic Weighted Round-Robin assigns fixed weights at startup and never changes them. DWRS recalculates weights continuously as:

- New packets arrive with different sizes (m_i updates every request)
- Server queues fill or empty (affecting eligibility)
- The system adapts in real-time to actual traffic conditions

DWRS vs other algorithms — quick view:

Algorithm	Full Name	Dynamic	Weighted	Round-Robin	Dynamic
RR	Round-Robin	No	No	No	No
WRR	Weighted RR	Yes	No	No	No
DWRR	Deficit Weighted RR	Yes	Yes	No	No
WLB	Weighted Least Busy	No	Yes	Yes	No
DWRS	Dynamic Weighted RSS	Yes	Yes	Yes	No
HDW	Hash+DWRS+DSP	Yes	Yes	Yes	Yes

3. Hash-IP — Fingerprinting Every Request

Every request that enters the HDW system gets a **unique fingerprint** (hash key) before any scheduling happens. This fingerprint is computed from 4 inputs combined together:

RN(i) Request Number Monotonic counter — prevents replay attacks	srcIP dstIP IP Pair Who sent it and who it's going to	Data Packet Size (KB) How big the packet is	Weight w_i pktSize / 1500 Relative heaviness vs standard MTU
--	---	---	---

All four inputs are combined and fed into **FNV-1a hashing** to produce an 8-character hex key like **3A7F9C12**.

Why hash at all?

Purpose 1: Security The hash acts like a packet ID card. Any altered packet (different IP, different size) produces a completely different hash. This detects tampering, IP spoofing, and replay attacks where an attacker re-sends an old packet.	Purpose 2: Lookup Speed The hash is stored in a hash table. Later, if we need to find information about this request quickly (e.g. for routing decisions), we can look it up in $O(1)$ constant time instead of searching through all requests.
--	---

FNV-1a Hash — Why this algorithm?

FNV-1a (Fowler-Noll-Vo 1a) is a non-cryptographic hash function chosen for: • Speed — extremely fast, just XOR and multiply operations • Low collision rate — very unlikely two different inputs produce the same hash • Avalanche effect — one bit change in input flips ~50% of output bits, making tampering obvious Alternatives considered: MD5 (too slow for per-packet use), SHA-256 (even slower), CRC32 (too many collisions). FNV-1a wins for network packet use.
--

The Hash Table — Storing and retrieving entries:

Once the hash key is computed, it is stored in a **1024-slot hash table** using **open addressing with linear probing**:

Term	Explanation
Insert	Compute slot = hexKey % 1024. If occupied, try slot+1, slot+2... until empty slot found. Store entry.
Collision	When two keys map to the same slot. Resolved by linear probing (try next slot). Collision counter increments.
Search	Start at slot = hexKey % 1024. Walk forward until key matches or null found.
Delete (lazy)	Mark slot as DELETED (not null). This preserves the probing chain so later lookups don't short-circuit.
TTL eviction	After 2500ms, each entry is deleted. Keeps table fresh — old requests don't linger forever.
Why 1024 slots?	Power of 2 — makes the modulo operation very fast (bit masking). Large enough to avoid too many collisions.

4. DSP — Dynamic Switching Path (IGRP / EGRP)

Even the best scheduling algorithm sometimes picks a server that is congested by the time the packet arrives. The **Dynamic Switching Path (DSP)** module is a safety net — it reroutes packets away from overloaded servers.

The Zone System:

Servers are split into two zones at startup:

Inner Zone (IGRP) = first half of servers (e.g. Server A, Server B)

Outer Zone (EGRP) = second half of servers (e.g. Server C)

With 3 servers: inner = [A, B], outer = [C]. With 4 servers: inner = [A, B], outer = [C, D].

The 4-Step Decision Tree (for every request):

Step	Condition	Action
1	Target server load <= 85%	Use target server as-is Not congested — proceed normally
2	Target load > 85% Inner zone server available with load < 70%	Reroute via IGRP Keep traffic "local" first — inner servers are primary
3	Inner zone all > 70% Outer zone server available with load < 90%	Reroute via EGRP Escalate to outer zone as fallback
4	All servers > 90%	Send to original target anyway SATURATED — no good option, best effort

IGRP vs EGRP — terminology explained:

IGRP = Inner Gateway Routing Protocol (adapted)

In networking, IGRP routes traffic within the same autonomous system (same network zone). Here it means: try to reroute within the inner server zone first. Inner servers are typically primary, higher-priority machines.

EGRP = Exterior Gateway Routing Protocol (adapted)

In networking, EGRP routes between different autonomous systems. Here it means: escalate beyond the inner zone to outer servers. Outer servers are secondary/overflow machines used when inner zone is saturated.

Why the asymmetric thresholds (70% for inner, 90% for outer)?

Inner zone threshold is stricter (70%) because we prefer to keep inner servers somewhat free for new primary work. Outer zone accepts higher load (90%) because it is already the last resort — we accept higher risk there. The original target must be very congested (85%) before any rerouting happens at all.

5. HDW — The Complete Pipeline

HDW = **H**ash-IP + **D**ynamic Weighted Scheduler + **DSP** (Dynamic Switching Path). It combines all three components into one unified load balancing system. Every incoming request flows through exactly 4 steps:

Step 1 Hash-IP Fingerprint the request using FNV-1a on srcIP, dstIP, weight, pktSize, RN	Step 2 Weighted Scheduler Select best server using deficit scoring (Eq.1–4). Queue-full servers excluded.	Step 3 DSP Check Is selected server congested (>85%)? If yes, reroute via IGRP or EGRP.	Step 4 Process Server handles packet. Update jitter, throughput, queue depth, meanPacketSize.
--	---	---	---

Each step feeds into the next. The hash from Step 1 is stored for audit. The server chosen in Step 2 may be swapped in Step 3. All metrics are updated in Step 4.

What makes HDW better than plain DWRS?

Feature	How HDW adds value over plain DWRS
Security layer	DWRS alone has no packet authentication. HDW adds Hash-IP so tampered/spoofed packets are rejected.
Congestion escape	DWRS schedules well under normal load but can still route to a momentarily-congested server.
Audit trail	Every packet gets a hash key stored in the hash table. Enables forensic lookup and integrity verification.
Adaptive weights	Both use dynamic weights, but HDW logs security events every 15 requests and verifies hash integrity.

The queue drain mechanism:

Every 400ms, a background timer runs and drains each server's queue proportionally:

Rate = max(1, round(server.maxThroughputMbps / minThroughput))

So if Server C has 1000 Mbps and Server A has 100 Mbps, and minThroughput = 100, then:

- Server A drains 1 request per tick
- Server C drains 10 requests per tick

This simulates real server processing speed differences.

6. The 4 Key Equations from the Paper

These equations are from *Osei Kofi & Ahene, PLOS ONE 2023*. They define how scheduling weights are computed.

Equation 1 — Proportional scheduling share

$$\text{proportion}_i = w_i / (w_1 + w_2 + \dots + w_n)$$

What fraction of all requests should server i get? Answer: its weight divided by the sum of all weights. If $w_A=1$, $w_B=2$, $w_C=3$, then A gets $1/6 = 16.7\%$, B gets $2/6 = 33.3\%$, C gets $3/6 = 50\%$ of requests.

Equation 2 — Bandwidth allocation share

$$\text{bw}_i = (m_i \times w_i) / \text{sum}_k(m_k \times w_k)$$

What share of actual BANDWIDTH (not just request count) does server i handle? Multiplies weight by mean packet size m_i to account for heavy vs light packets. A server handling small packets can process more of them for the same bandwidth.

Equation 3 — Link capacity share

$$l_i = \text{maxThroughput}_i / \text{sum}_k(\text{maxThroughput}_k)$$

What fraction of the total available bandwidth does server i own? Example: Server A=100 Mbps, B=500 Mbps, C=1000 Mbps. Total=1600 Mbps. $l_A=0.0625$, $l_B=0.3125$, $l_C=0.625$. These are raw capacity ratios before packet size is considered.

Equation 4 — Scheduling weight

$$w_i = l_i / m_i$$

The final scheduling weight used in Eq.1 and Eq.2. Divides link capacity by mean packet size. WHY divide by m_i ? Because if a server handles LARGE packets, it should get FEWER of them (each one uses more of its capacity). Smaller packets = server can handle more = higher weight.

How equations chain together:

Eq.3 computes l_i from raw Mbps data → Eq.4 divides l_i by m_i to get w_i → Eq.1 normalises w_i to get proportional request share → Eq.2 multiplies $m_i \times w_i$ to get actual bandwidth share.

7. Code Walkthrough — Every File Explained

core/models.js — Server & ClientRequest classes

Server class — represents one physical/virtual server with these key fields: • `maxThroughputMbps` = peak bandwidth (Eq.3 uses this) • `queueCapacity` = max concurrent requests in queue • `meanPacketSizeKB` = `m_i`, updated live via `updateMeanPacketSize()` • `queue` = current depth (requests waiting to be processed) • `jitter` = $|RT_current - RT_previous|$ — variation in response time **process(req) method** — simulates handling one request: $baseRT = (pktSize \times 8 \text{ bits}) / (maxThroughput \times 1000)$ [time to transmit] $congestionPenalty = (queue/capacity) \times baseRT \times 2$ [slowdown when busy] $responseTime = baseRT + congestionPenalty + \text{random noise}$ `jitter = |responseTime - prevResponseTime|` **ClientRequest class** — represents one incoming packet: `weight` = `pktSizeKB` / 1500 [1500 KB = standard MTU reference for Eq.4] `hashKey`, `assignedServer` filled in later by `HDWLoadBalancer.dispatch()`

core/hash-ip.js — FNV-1a hashing

HashIP.fnv1a(str) — core hash function: Start with seed: `h = 2166136261` (FNV offset basis) For each character: $h = (h \text{ XOR } charCode) \times 16777619$ (FNV prime) Returns 32-bit unsigned integer **HashIP.computeKey(srcIP, dstIP, weight, pktSize, requestNum)**: Concatenates: "srcIP|dstIP|weight|pktSize|requestNum" Feeds string into `fnv1a()` Returns 8-character uppercase hex (e.g. "3A7F9C12") **Why concatenate these 5 inputs?** Each adds uniqueness: `RN(i)` = prevents replay (same packet re-sent gets different hash) `srcIP+dstIP` = ties hash to the connection `pktSize` = accounts for packet size in the fingerprint `weight` = ensures even same-size packets from different contexts differ

core/hash-table.js — 1024-slot open-addressed hash table

insert(hexKey, payload): `slot = parseInt(hexKey, 16) % 1024` [map hex → array index] If slot occupied: try `slot+1`, `slot+2`... (linear probing) If table full (probes = 1024): return null (drop silently) **search(hexKey)**: Same starting slot calculation Walk forward until key matches OR null found Skips DELETED tombstone slots (keeps probing) **remove(hexKey)** — lazy deletion: Sets slot to string "DELETED" instead of null WHY? If we set to null, future searches might stop too early Example: `A→B→C` inserted (all collision). Remove B. If `B=null`, search for C starts at A, jumps to B (null), STOPS — never finds C! With tombstone DELETED, search continues past it to C. Correct. **Why 1024?** Power of 2 — modulo becomes bitwise AND, very fast. **TTL**: 2500ms after insertion, entry is evicted via `setTimeout`.

core/weighted-scheduler.js — Scheduling logic (Eq.1-4)

computeWeights(servers) — Eq.3 + Eq.4: For each server: $l_i = maxThroughputMbps / totalThroughput$ [Eq.3] Then: $w_i = l_i / meanPacketSizeKB$ [Eq.4] Returns array of `{id, li, wi}` **computeBandwidthAllocations(servers)** — Eq.2: `denom = sum( m_k × w_k )` for all servers For each server: $bw = (m_i \times w_i) / denom$ [Eq.2] Returns array of `{id, wi, li, bw}` **selectServerHDW(servers)** — Deficit Round-Robin: Each server accumulates `credit = w_i / totalW` per tick [Eq.1] Server with HIGHEST credit wins → gets request Winner's credit DECREASES by 1.0 (the "deficit") Queue-full servers excluded from eligibility **selectServer(servers, mode)** — routes to correct algorithm: `hdw` → `selectServerHDW` (deficit RR with weights) `dwrr` → `_dwrr` (slot-based round-robin with weight multiplier) `wlb` → least-queue server (ignores weights) `dwr` → random server (pure random, for comparison)

core/dynamic-switching-path.js — Rerouting (IGRP/EGRP)

_buildZones(servers): inner = servers.slice(0, ceil(n/2)) outer = servers.slice(ceil(n/2)) 3 servers → inner=[A,B], outer=[C] **route(target, allServers)** — the decision tree: loadRatio = target.queue / target.queueCapacity if loadRatio <= 0.85: return target (not congested) else: find least-loaded inner server NOT equal to target if innerAlt.loadRatio < 0.70: reroute IGRP to innerAlt else: find least-loaded outer server if outerAlt.loadRatio < 0.90: reroute EGRP to outerAlt else: return target with reason SATURATED **Note:** DSP is ONLY active in "hdw" mode. In dwrr/wlb/dwrs modes, route() is skipped entirely — the scheduled server is used directly. This is intentional: those modes are simpler baselines for comparison.

core/hdw-load-balancer.js — Main orchestrator

constructor(serverConfigs, mode): Creates HashTable (1024 slots) Creates Server objects from configs Creates DSP with those servers Starts queue drain timer (every 400ms) Calls WeightedScheduler.reset() to init deficit counters **dispatch(minPkt, maxPkt)** — the 4-step pipeline: Step 1: Generate srcIP, dstIP, pktSize. Compute hash. Store in hashTable. Step 2: WeightedScheduler.selectServer() picks target server. Step 3: If mode=hdw, DSP.route() may reroute to different server. Step 4: Increment queue. Call server.process(req). Update totals. Every 15 requests (HDW mode): log hash table integrity check. After 2500ms: hashTable.remove(hashKey) [TTL eviction] **_drainQueues()** — called every 400ms: minT = minimum throughput across all servers Each server drains: queue -= max(1, round(server.maxThroughput / minT)) **Lifecycle: destroy() / reset():** destroy() stops the drain timer (prevents memory leaks) reset() calls destroy, resets all counters, restarts timer

8. Algorithm Comparison — Full Reference

The codebase implements 4 modes selectable at runtime. Here is exactly what each does:

Mode	Description	Throughput/min	Equations	Security
hdw	Deficit Round-Robin with w_i credits (Eq.1–4). Highest credit wins	Yes	Yes	Deficit after full security + smart routing
dwrr	Slot array: server gets N slots proportional to $\max(\text{Throughput}/\text{min}$	No	Yes	Simple weighted distribution, no congestion escape
wlb	Always pick server with smallest queue. Ignores all	No	Yes	Pure least-connections — ignores server capacity
dwr	Pick a completely random server from the pool.	No	Yes	Baseline/testing only — worst algorithm

Key metric differences between modes:

- **HDW**: best jitter (steady response times), most reroutes (proactive), highest security, most CPU overhead
- **DWRR**: good distribution, moderate jitter, no congestion escape
- **WLB**: lowest queue depths (greedy least-connections), ignores bandwidth differences
- **DWRS**: worst overall — exists only as a performance baseline to show how bad random is

9. Scenario Q&A; — Exam-Style Questions

Use this section to test your understanding. Each question covers a specific concept or scenario.

Q1: Server A has 100 Mbps, B has 500 Mbps, C has 1000 Mbps. All have mean packet size 750 KB. Compute I_i and w_i for each server.

- Total throughput = $100 + 500 + 1000 = 1600$ Mbps
- $I_A = 100/1600 = 0.0625$ | $I_B = 500/1600 = 0.3125$ | $I_C = 1000/1600 = 0.625$
- $w_A = 0.0625/750 = 0.0000833$ | $w_B = 0.3125/750 = 0.000417$ | $w_C = 0.625/750 = 0.000833$
- Ratio $w_A : w_B : w_C = 1 : 5 : 10$
- So Server C gets ~10x more requests than Server A. Correct — it's 10x faster.

Q2: Server B is selected as target. Its queue is 140/150 (93.3%). Inner zone has Server A at 60% load. What does DSP do?

- loadRatio of B = $140/150 = 0.933$ → exceeds 0.85 threshold → DSP activates.
- Check inner zone: Server A is at 60% load → $60\% < 70\%$ threshold → ELIGIBLE for IGRP reroute.
- Decision: REROUTE via IGRP to Server A.
- Result: req goes to Server A, not B. DSP.reroutes counter increments by 1.
- Log shows: "DSP IGRP: Server B → Server A | Server B load: 93%"

Q3: Why is lazy deletion (DELETED tombstone) used in the hash table instead of setting null?

- Linear probing creates "chains" of entries that collided at the same slot.
- If we delete by setting null, the chain is broken — future searches stop at that null.
- Example: keys X, Y, Z all hash to slot 10. Y is stored at 11, Z at 12.
- Delete Y → set slot 11 to null. Now search for Z: hash→10, check 10 (not Z),
- check 11 (NULL) → STOP. Z is declared "not found" even though it's at 12!
- With tombstone DELETED: check 10 (not Z), check 11 (DELETED, skip), check 12 → FOUND Z. Correct.

Q4: What happens if all 1024 hash table slots are full?

- `insert()` probes all 1024 slots without finding an empty one.
- When probes $\geq$ size (1024): return null — the insert silently fails.
- The request is still processed (scheduling continues), but no hash record is stored.
- This is acceptable: the hash table is for security audit, not for routing decisions.
- In practice this rarely happens: entries are evicted after 2500ms TTL,
- so slots are continuously freed. With 1024 slots and ~10 req/sec, load stays well below capacity.

Q5: A student says "DWRS mode picks a random server." Is that correct?

- INCORRECT. In this codebase, mode "dwrs" is implemented as random selection (`Math.random()`).
- This is intentional — it is a BASELINE comparison mode showing how bad random routing is.
- The ACTUAL DWRS algorithm (Dynamic Weighted Round-Robin Scheduling) is what runs in HDW mode.
- In HDW mode, the weighted scheduler uses Eq.1–4 — that is the true DWRS implementation.
- The naming is counterintuitive: "dwrs" mode = random, "hdw" mode = full DWRS + Hash + DSP.

Q6: Explain jitter. Why does the HDW algorithm aim to minimise it?

- Jitter = $|\text{responseTime_current} - \text{responseTime_previous}|$ for a given server.
- High jitter means response times are erratic — sometimes 5ms, sometimes 200ms.
- For real-time applications (video calls, gaming, VoIP), jitter causes glitches.
- HDW minimises jitter by distributing load smoothly: no server gets suddenly overwhelmed,
- so response times stay consistent. The deficit RR system prevents any server from receiving
- a burst of requests far above its fair share, keeping queue depth steady.
- DSP further helps: if a server is congested (which causes high response time spikes),
- DSP reroutes to a less-busy server, preventing jitter spikes at the congested server.

Q7: Server C has meanPacketSizeKB = 2000. Server A has meanPacketSizeKB = 100. Both have same $I_i = 0.5$. Which server gets more requests and why?

- $w_A = I_A / m_A = 0.5 / 100 = 0.005$
- $w_C = I_C / m_C = 0.5 / 2000 = 0.00025$
- Server A has $w_A/w_C = 20x$ higher weight.
- Server A gets 20x more requests than Server C.
- WHY? Server C processes very large packets (2000 KB each). Each packet uses more bandwidth.
- Even though it has the same raw capacity (I_i), it can handle fewer packets per second.
- So A, which processes small packets (100 KB), gets far more requests. This is Eq.4 in action.

Q8: Trace a request through the complete HDW pipeline step by step.

- 1. Request arrives: srcIP=192.168.1.5, dstIP=10.0.0.2, pktSize=800 KB, RN=42
- 2. Hash-IP: raw="192.168.1.5|10.0.0.2|0.53333|800|42" → FNV-1a → "A3C7F812"
- 3. HashTable.insert("A3C7F812", {rn:42, srcIP, dstIP, weight, data}) → stored at slot 532
- 4. WeightedScheduler: computes w_i for all servers, adds credits (Eq.1), picks Server C (highest credit)
- 5. Server C credit decreases by 1.0
- 6. DSP: Server C load = $40/300 = 13\%$ → well below 85% → NO reroute, use Server C as-is
- 7. Server C: queue++, process(req) called, meanPacketSizeKB updated, jitter calculated
- 8. globalThroughput += 800/1024 MB, globalJitter += server.jitter
- 9. After 2500ms: hashTable.remove("A3C7F812") → slot marked DELETED

Quick Reference Summary

One-line definition	
DWRS	Dynamic Weighted Round-Robin Scheduling — assigns servers based on capacity and packet size
HDW	Hash + Dynamic Weighted Scheduler + DSP — full pipeline combining all three modules
Hash-IP	Fingerprints every packet using FNV-1a on srcIP, dstIP, pktSize, weight, RN

FNV-1a	Fast non-cryptographic hash: XOR each byte with running value, multiply by FNV prime
Hash table	1024-slot array with linear probing and lazy (tombstone) deletion
Eq.3 l_i	Link capacity share = $\text{maxThroughput}_i / \text{totalThroughput}$
Eq.4 w_i	Scheduling weight = l_i / m_i . Higher w_i = more requests assigned
Eq.1 proportion	Request share = $w_i / \text{sum}(\text{all } w_k)$
Eq.2 bandwidth	BW share = $(m_i \times w_i) / \text{sum}(m_k \times w_k)$
DSP	Reroutes congested requests: IGRP (inner zone) then EGRP (outer zone)
IGRP	Inner zone reroute — threshold: target > 85%, inner alt < 70% load
EGRP	Outer zone reroute — threshold: inner all $\geq$ 70%, outer alt < 90% load
Jitter	$ \text{RT}_{\text{current}} - \text{RT}_{\text{previous}} $ — variation in response time per server
Deficit RR	Each server accumulates credit per tick; highest credit wins; credit depleted after win
Queue drain	Background timer every 400ms reduces server queue proportional to its throughput
TTL	2500ms after insertion, hash entry evicted to keep table fresh
Linear probing	Collision resolution: if slot N taken, try N+1, N+2, ... until empty slot found

Paper reference: Osei Kofi & Ahene, "HDW Load Balancer: A Hybrid Dynamic Weighted Scheduling Algorithm for Network Load Balancing," PLOS ONE 18(4), 2023.

This PDF covers all theory, equations, code structure, and scenario questions needed to answer any exam question on DWRS and HDW load balancing.